CLOUD
DATA ENGINEER

Name: S A THARUN
USN: 23BTRCF026
Batch: CSE-CTIS
HCL ID: HCLTFP2283977
College: Jain University

SECTION A: ADVANCED SQL

Question 1: Database Design and Advanced Queries

1.Create a database named EnterpriseDB

```
1    CREATE DATABASE EnterpriseDB;
2
3 •  USE EnterpriseDB;
4
```

2.Created Table Employees

```
CREATE TABLE Employees (
    EmployeeID INT PRIMARY KEY,
    EmployeeName VARCHAR(50),
    DepartmentID INT,
    Salary DECIMAL(10,2),
    JoiningDate DATE,
    ManagerID INT,
    FOREIGN KEY (DepartmentID) REFERENCES Departments(DepartmentID),
    FOREIGN KEY (ManagerID) REFERENCES Employees(EmployeeID)
);
```

3.Created Table Departments

```
7 • CREATE TABLE Departments (
8       DepartmentID INT PRIMARY KEY,
9       DepartmentName VARCHAR(50),
10      Location VARCHAR(50)
11  );
```

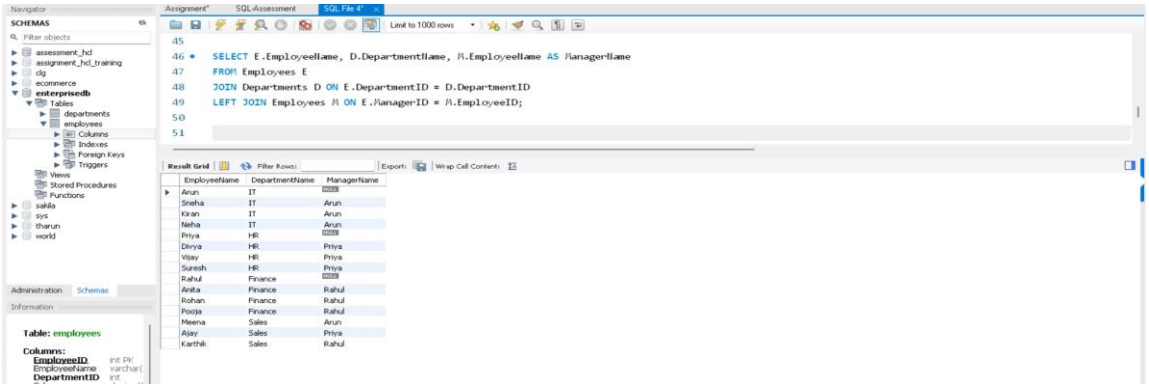
4.Inserted Values Both in Employees and Department Table

```sql
INSERT INTO Departments VALUES
(1, 'IT', 'Chennai'),
(2, 'HR', 'Bangalore'),
(3, 'Finance', 'Mumbai'),
(4, 'Sales', 'Delhi'),
(5, 'Marketing', 'Hyderabad');

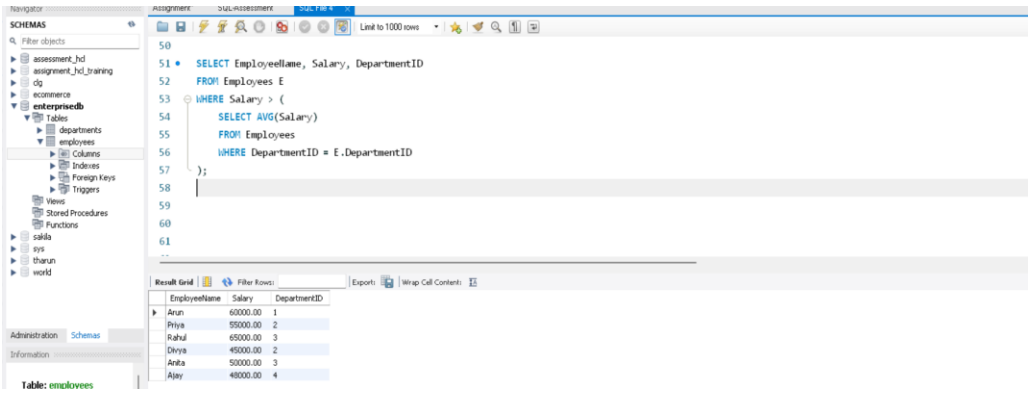
INSERT INTO Employees VALUES
(101, 'Arun', 1, 60000, '2022-01-15', NULL),
(102, 'Priya', 2, 55000, '2021-03-10', NULL),
(103, 'Rahul', 3, 65000, '2020-05-12', NULL),
(104, 'Sneha', 1, 40000, '2023-06-12', 101),
(105, 'Kiran', 1, 35000, '2023-02-05', 101),
(106, 'Divya', 2, 45000, '2021-11-18', 102),
(107, 'Vijay', 2, 32000, '2022-04-22', 102),
(108, 'Anita', 3, 50000, '2023-07-14', 103),
(109, 'Rohan', 3, 30000, '2024-01-10', 103),
(110, 'Meena', 4, 42000, '2024-02-15', 101),
(111, 'Ajay', 4, 48000, '2022-09-12', 102),
(112, 'Neha', 1, 36000, '2023-05-25', 101),
(113, 'Suresh', 2, 41000, '2022-12-01', 102),
(114, 'Pooja', 3, 34000, '2024-03-18', 103),
```

Tasks:

a. Display employee names, department names, and manager names using joins.



b. Find employees earning more than their department's average salary.



c. Find the second-highest distinct salary without using LIMIT, TOP, or FETCH FIRST.

```
59 •  SELECT MAX(Salary)
60    FROM Employees
61    WHERE Salary < (SELECT MAX(Salary) FROM Employees);
62
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

MAX(Salary)
60000.00

d. Rank employees by salary within each department and display the top three employees per department, retaining ties at the cutoff. (2 marks)

```
63 •  SELECT EmployeeName, DepartmentID, Salary
64    FROM (
65        SELECT EmployeeName, DepartmentID, Salary,
66               RANK() OVER (PARTITION BY DepartmentID ORDER BY Salary DESC) AS R
67        FROM Employees
68    ) AS T
69    WHERE R <= 3;
70
71
72
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

EmployeeName	DepartmentID	Salary
Arun	1	60000.00
Sneha	1	40000.00
Neha	1	36000.00
Priya	2	55000.00
Divya	2	45000.00
Suresh	2	41000.00
Rahul	3	65000.00
Anita	3	50000.00
Pooja	3	34000.00
Ajay	4	48000.00
Meena	4	42000.00
Karthik	4	39000.00

Question 2: Sales Analytics and Query Optimization (6 Marks)

```sql
-- TABLE SALES

CREATE TABLE Sales (
    SaleID INT PRIMARY KEY,
    CustomerID INT,
    SaleDate DATE,
    ProductName VARCHAR(50),
    Quantity INT,
    UnitPrice DECIMAL(10,2)
);

INSERT INTO Sales VALUES
(1, 101, '2025-01-05', 'Laptop', 2, 50000),
(2, 102, '2025-01-10', 'Mouse', 5, 500),
(3, 103, '2025-01-15', 'Keyboard', 3, 1000),
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

SaleID	CustomerID	SaleDate	ProductName	Quantity	UnitPrice
1	101	2025-01-05	Laptop	2	50000.00
2	102	2025-01-10	Mouse	5	500.00
3	103	2025-01-15	Keyboard	3	1000.00
4	101	2025-01-20	Monitor	2	12000.00
5	104	2025-02-03	Laptop	1	50000.00
6	102	2025-02-08	Mouse	10	500.00
7	105	2025-02-12	Keyboard	4	1000.00
8	103	2025-02-18	Monitor	3	12000.00
9	101	2025-03-02	Laptop	2	50000.00

a. Calculate total revenue for each product. (2 marks)

```sql
SELECT ProductName, SUM(Quantity * UnitPrice) AS TotalRevenue
FROM Sales
GROUP BY ProductName;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ProductName	TotalRevenue
Laptop	300000.00
Mouse	14500.00
Keyboard	14000.00
Monitor	72000.00

b. Calculate monthly revenue and month-over-month revenue change using a window function. (2 marks)

```sql
SELECT Month, Revenue,
       Revenue - LAG(Revenue) OVER (ORDER BY Month) AS RevenueChange
FROM (
    SELECT DATE_FORMAT(SaleDate, '%Y-%m') AS Month,
           SUM(Quantity * UnitPrice) AS Revenue
    FROM Sales
    GROUP BY DATE_FORMAT(SaleDate, '%Y-%m')
) AS T;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Month	Revenue	RevenueChange
2025-01	129500.00	NULL
2025-02	95000.00	-34500.00
2025-03	120000.00	25000.00
2025-04	56000.00	-64000.00

c. Rewrite the following query to improve the potential for index usage and explain why your version may perform better. (2 marks)

SELECT * FROM Employees WHERE YEAR(JoiningDate) = 2025;

```
SELECT *
FROM Employees
WHERE JoiningDate >= '2025-01-01'
AND JoiningDate < '2026-01-01';
```

The above query checks a date range instead of using the YEAR () function. If an index exists on Joining Date, the database can use it more effectively to find employees who joined in 2025.

Question 3: SQL Troubleshooting

a. Explain how an execution plan helps identify expensive operations in a query.

An execution plan shows how the database runs a query. It helps identify slow operations, such as full table scans, unnecessary joins, and costly sorting. By checking the execution plan, we can find problems and improve the query performance.

b. Recommend an appropriate index strategy for queries filtering employees by DepartmentID and Salary. Explain your reasoning.

CREATE INDEX idx_dept_salary

ON Employees (DepartmentID, Salary);

The reason for this approach is that this index helps the database find employees based on their department and salary more quickly. It can reduce the amount of data the database needs to search. Henceforth Increasing the performance and Efficiency.

c. Write a query to identify duplicate customer email addresses and display

only addresses occurring more than once.

```
SELECT Email, COUNT(*)
FROM Customers
GROUP BY Email
HAVING COUNT(*) > 1;
```

The above query groups customers by email address and displays only the email addresses that appear more than once.

SECTION B: LINUX

Question 4: Log Analysis

a. Find all lines containing ERROR.

```
tharun@Tharun:/mnt/c/Users/91833$ nano application.log
tharun@Tharun:/mnt/c/Users/91833$ grep "ERROR" application.log
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 11:05:12 ERROR login-service Login failed
tharun@Tharun:/mnt/c/Users/91833$
```

b. Display the 20 most recent error entries in the file.

```
tharun@Tharun:/mnt/c/Users/91833$ grep "ERROR" application.log | tail -n 20
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 11:05:12 ERROR login-service Login failed
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 11:05:12 ERROR login-service Login failed
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 11:05:12 ERROR login-service Login failed
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 11:05:12 ERROR login-service Login failed
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 11:05:12 ERROR login-service Login failed
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 11:05:12 ERROR login-service Login failed
tharun@Tharun:/mnt/c/Users/91833$
```

c. Count the total number of error entries.

```
tharun@Tharun:/mnt/c/Users/91833$ grep -c "ERROR" application.log
12
tharun@Tharun:/mnt/c/Users/91833$
```

d. Extract error entries for a specified date into a separate file.

```
tharun@Tharun:/mnt/c/Users/91833$ grep '2026-10-28.*ERROR' application.log > errors.txt
tharun@Tharun:/mnt/c/Users/91833$ cat errors.txt
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 11:05:12 ERROR login-service Login failed
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 11:05:12 ERROR login-service Login failed
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 11:05:12 ERROR login-service Login failed
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 11:05:12 ERROR login-service Login failed
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 11:05:12 ERROR login-service Login failed
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 11:05:12 ERROR login-service Login failed
tharun@Tharun:/mnt/c/Users/91833$
```

e. Explain how you would calculate the number of errors occurring in each hour.

```
tharun@Tharun:/mnt/c/Users/91833$ grep "ERROR" application.log | cut -c 1-13 | sort | uniq -c
      6 2026-10-28 10
      6 2026-10-28 11
```

This command finds all lines containing ERROR, extracts the date and hour from each line, and sorts them. Finally, uniq -c counts how many errors occurred during each hour. This helps us easily identify the hours when the most errors occurred.

Question 5: Permissions and System Troubleshooting

a. Create a group named finance and a report file.

#Creates a Group named Finance

sudo groupadd finance

#Creates a file named Touch

touch report.txt

```
Microsoft Windows [Version 10.0.26300.9711]
(c) Microsoft Corporation. All rights reserved.

C:\Users\91833>wsl -d ubuntu
tharun@Tharun:/mnt/c/Users/91833$ sudo groupadd finance
[sudo: authenticate] Password:
tharun@Tharun:/mnt/c/Users/91833$ touch report.txt
tharun@Tharun:/mnt/c/Users/91833$
```

b. Set permissions so the owner can read and write, the finance group can read, and other users have no access.

#Assigns the file the finance group

sudo chgrp finance report.txt

#Set Permissions

chmod 640 report.txt

```
tharun@Tharun:/mnt/c/Users/91833$ sudo chgrp finance report.txt
tharun@Tharun:/mnt/c/Users/91833$ chmod 640 report.txt
tharun@Tharun:/mnt/c/Users/91833$
```

c. Identify the processes consuming the most CPU.

top

```
tharun@Tharun:/mnt/c/Users/91833$ top
top - 06:55:09 up 9 min,  2 users,  load average: 0.00, 0.14, 0.12
Tasks:  29 total,   1 running,  28 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   2879.1 total,   2088.5 free,    422.4 used,    442.5 buff/cache
MiB Swap:   1024.0 total,   1024.0 free,      0.0 used.   2456.8 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
    727 tharun    20   0    8180   5904   3768 R   1.1   0.2   0:00.04 top
      1 root      20   0   24312  14992  11188 S   0.0   0.5   0:03.39 systemd
      2 root      20   0    3172   2208   2072 S   0.0   0.1   0:00.05 init-systemd(Ub
      7 root      20   0    3172   2044   1940 S   0.0   0.1   0:00.00 init
     62 root      19  -1   50380  16100  14804 S   0.0   0.5   0:00.61 systemd-journal
     95 systemd+  20   0   22524  14224  11608 S   0.0   0.5   0:00.42 systemd-resolve
    108 root      20   0   35096  11932   8964 S   0.0   0.4   0:00.87 systemd-udevd
    141 root      20   0    2880   1916   1792 S   0.0   0.1   0:00.16 chronyd-starter
    149 root      20   0    4464   2980   2732 S   0.0   0.1   0:00.03 cron
    152 message+  20   0    8800   5204   4548 S   0.0   0.2   0:00.14 dbus-daemon
    160 root      20   0   44076  29296  14168 S   0.0   1.0   0:00.58 networkd-dispat
    164 root      20   0   18560   9308   8044 S   0.0   0.3   0:00.26 systemd-logind
    166 root      20   0 1866024  15468  12768 S   0.0   0.5   0:00.31 wsl-pro-service
    172 syslog    20   0  220532   5296   4368 S   0.0   0.2   0:00.16 rsyslogd
    222 _chrony   20   0   20404   6148   5380 S   0.0   0.2   0:00.08 chronyd
    231 root      20   0  123440  32152  17552 S   0.0   1.1   0:00.47 unattended-upgr
    232 _chrony   20   0   12076   2432   1792 S   0.0   0.1   0:00.01 chronyd
    235 root      20   0    5484   2596   2412 S   0.0   0.1   0:00.02 agetty
    342 root      20   0    8484   5316   4496 S   0.0   0.2   0:00.03 login
    388 root      20   0   22492  13184  10744 S   0.0   0.4   0:00.39 systemd
    390 root      20   0   22904   3984   1992 S   0.0   0.1   0:00.00 (sd-pam)
    418 root      20   0    6128   5300   3792 S   0.0   0.2   0:00.06 bash
    539 root      20   0    3176   1120    980 S   0.0   0.0   0:00.00 SessionLeader
    540 root      20   0    3192   1260   1108 S   0.0   0.0   0:00.03 Relay(541)
    541 tharun    20   0    6260   5472   3804 S   0.0   0.2   0:00.11 bash
    546 root      20   0    8632   5360   4524 S   0.0   0.2   0:00.02 login
```

d. Display memory usage and filesystem disk utilization.

```
tharun@Tharun:/mnt/c/Users/91833$ free -h
               total        used        free      shared  buff/cache   available
Mem:           2.8Gi       458Mi       2.3Gi       3.5Mi       183Mi       2.4Gi
Swap:          1.0Gi          0B       1.0Gi
```

```
tharun@Tharun:/mnt/c/Users/91833$ df -h
Filesystem      Size  Used Avail Use% Mounted on
none            1.5G     0  1.5G   0% /usr/lib/modules/6.6.114.1-microsoft-standard-WSL2
none            1.5G  4.0K  1.5G   1% /mnt/wsl
drivers         238G  122G  116G  52% /usr/lib/wsl/drivers
/dev/sdd       1007G  1.5G  955G   1% /
none            1.5G   40K  1.5G   1% /mnt/wslg
none            1.5G     0  1.5G   0% /usr/lib/wsl/lib
rootfs          1.5G  2.8M  1.4G   1% /init
none            1.5G  520K  1.5G   1% /run
none            1.5G     0  1.5G   0% /run/lock
none            1.5G     0  1.5G   0% /run/shm
none            1.5G  100K  1.5G   1% /mnt/wslg/versions.txt
none            1.5G  100K  1.5G   1% /mnt/wslg/doc
C:\             238G  122G  116G  52% /mnt/c
D:\             932G  283G  650G  31% /mnt/d
tmpfs           1.5G     0  1.5G   0% /tmp
none            1.0M     0  1.0M   0% /run/credentials/systemd-journald.service
none            1.0M     0  1.0M   0% /run/credentials/systemd-resolved.service
none            1.0M     0  1.0M   0% /run/credentials/getty@tty1.service
tmpfs           288M   12K  288M   1% /run/user/0
tmpfs           288M   12K  288M   1% /run/user/1000
```

e. Find files larger than 100 MB under a specified application directory.

```
tharun@Tharun:/mnt/c/Users/91833$ mkdir -p application
tharun@Tharun:/mnt/c/Users/91833$ truncate -s 101M application/testfile.txt
tharun@Tharun:/mnt/c/Users/91833$ find application -type f -size +100M
application/testfile.txt
tharun@Tharun:/mnt/c/Users/91833$
```

Question 6: Bash Scripting

1. Accepts source and destination directories as command-line arguments.

Script

```
tharun@Tharun:/mnt/c/Users/91833$ nano backup_check.sh
tharun@Tharun:/mnt/c/Users/91833$ cat backup_check.sh
#!/bin/bash

if [ $# -ne 2 ]; then
    echo "Please enter source and destination directories"
    exit 1
fi

if [ ! -d "$1" ]; then
    echo "Source directory does not exist"
    exit 1
fi

mkdir -p "$2"

if tar -czf "$2/backup_$(date +%Y%m%d_%H%M%S).tar.gz" -C "$1" .; then
    echo "Backup successful"
else
    echo "Backup failed"
fi
tharun@Tharun:/mnt/c/Users/91833$
```

Valid arguments

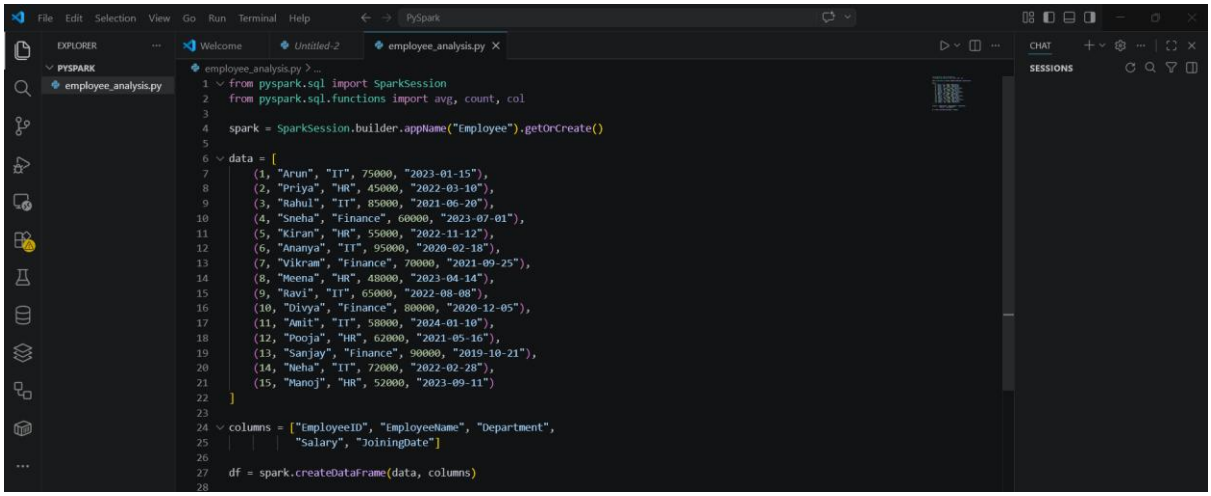
```
tharun@Tharun:/mnt/c/Users/91833$ chmod +x backup_check.sh
mkdir -p source
echo "Sample report" > source/report.txt
tharun@Tharun:/mnt/c/Users/91833$ ./backup_check.sh source backups
Backup successful
tharun@Tharun:/mnt/c/Users/91833$ ls backups
backup_20261028_083239.tar.gz  backup_20261028_083556.tar.gz  backup_20261028_084302.tar.gz
tharun@Tharun:/mnt/c/Users/91833$
```

Invalid arguments

```
tharun@Tharun:/mnt/c/Users/91833$ ./backup_check.sh source
Please enter source and destination directories
tharun@Tharun:/mnt/c/Users/91833$ ./backup_check.sh wrong backups
Source directory does not exist
tharun@Tharun:/mnt/c/Users/91833$
```

SECTION C
PYSPARK DATA ENGINEERING

Question 7: PySpark DataFrame Operations



a. Display the schema and first five records.

```
28  df = spark.createDataFrame(data, columns)
29
30
31  df.printSchema()
32  df.show(5)
33
34
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS python

```
PS D:\JGI\HCL TRAINING\HCL Assessment\PySpark> python employee_analysis.py
cable
root
 |-- EmployeeID: long (nullable = true)
 |-- EmployeeName: string (nullable = true)
 |-- Department: string (nullable = true)
 |-- Salary: long (nullable = true)
 |-- JoiningDate: string (nullable = true)

+----------+------------+----------+------+-----------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|
+----------+------------+----------+------+-----------+
|         1|        Arun|        IT| 75000| 2023-01-15|
|         2|       Priya|        HR| 45000| 2022-03-10|
|         3|       Rahul|        IT| 85000| 2021-06-20|
|         4|       Sneha|   Finance| 60000| 2023-07-01|
|         5|       Kiran|        HR| 55000| 2022-11-12|
```

b. Filter employees whose monthly salary is greater than ₹60,000.

```
33
34
35  df.filter(col("Salary") > 60000).show()
36
37
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell

```
PS D:\JGI\HCL TRAINING\HCL Assessment\PySpark> python employee_analysis.py
+----------+------------+----------+------+-----------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|
+----------+------------+----------+------+-----------+
|         1|        Arun|        IT| 75000| 2023-01-15|
|         3|       Rahul|        IT| 85000| 2021-06-20|
|         6|      Ananya|        IT| 95000| 2020-02-18|
|         7|      Vikram|   Finance| 70000| 2021-09-25|
|         9|       Ravi|        IT| 65000| 2022-08-08|
|        10|       Divya|   Finance| 80000| 2020-12-05|
|        12|       Pooja|        HR| 62000| 2021-05-16|
|        13|      Sanjay|   Finance| 90000| 2019-10-21|
|        14|       Neha|        IT| 72000| 2022-02-28|
+----------+------------+----------+------+-----------+
```

c. Add an AnnualSalary column calculated from the monthly salary.

```
38    df = df.withColumn("AnnualSalary", col("Salary") * 12)
39    df.show()
40
41
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\JGI\HCL TRAINING\HCL Assessment\PySpark> python employee_analysis.py

+----------+------------+----------+------+-----------+------------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|AnnualSalary|
+----------+------------+----------+------+-----------+------------+
|         1|        Arun|        IT| 75000| 2023-01-15|      900000|
|         2|       Priya|        HR| 45000| 2022-03-10|      540000|
|         3|       Rahul|        IT| 85000| 2021-06-20|     1020000|
|         4|       Sneha|   Finance| 60000| 2023-07-01|      720000|
|         5|       Kiran|        HR| 55000| 2022-11-12|      660000|
|         6|      Ananya|        IT| 95000| 2020-02-18|     1140000|
|         7|      Vikram|   Finance| 70000| 2021-09-25|      840000|
|         8|       Meena|        HR| 48000| 2023-04-14|      576000|
|         9|        Ravi|        IT| 65000| 2022-08-08|      780000|
|        10|       Divya|   Finance| 80000| 2020-12-05|      960000|
|        11|        Amit|        IT| 58000| 2024-01-10|      696000|
|        12|       Pooja|        HR| 62000| 2021-05-16|      744000|
|        13|      Sanjay|   Finance| 90000| 2019-10-21|     1080000|
|        14|        Neha|        IT| 72000| 2022-02-28|      864000|
|        15|       Manoj|        HR| 52000| 2023-09-11|      624000|
+----------+------------+----------+------+-----------+------------+
```

d. Group employees by department and calculate the average salary and employee count.

```
42  df.groupBy("Department").agg(
43      avg("Salary").alias("AverageSalary"),
44      count("*").alias("EmployeeCount")
45  ).show()
46
47
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell

```
PS D:\JGI\HCL TRAINING\HCL Assessment\PySpark> python employee_analysis.py
|        15|       Manoj|        HR| 52000| 2023-09-11|      624000|
+----------+------------+----------+------+-----------+------------+

+----------+-------------+-------------+
|Department|AverageSalary|EmployeeCount|
+----------+-------------+-------------+
|        HR|      52400.0|            5|
|        IT|      75000.0|            6|
|   Finance|      75000.0|            4|
+----------+-------------+-------------+
```

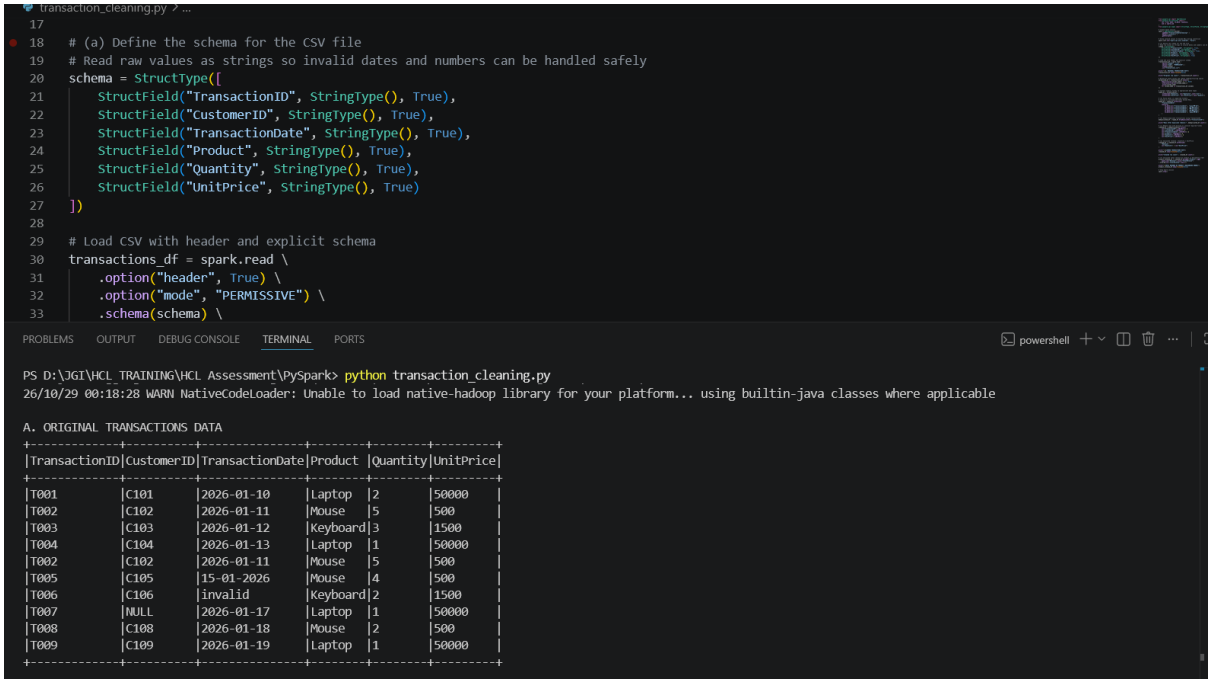
e. Display the five highest-paid employees.

employee_analysis.py > ...

```
47
48    df.orderBy(col("Salary").desc()).show(5)
49
50    spark.stop()
51
```

```
+----------+------------+----------+------+-----------+------------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|AnnualSalary|
+----------+------------+----------+------+-----------+------------+
|         6|      Ananya|        IT| 95000| 2020-02-18|     1140000|
|        13|      Sanjay|   Finance| 90000| 2019-10-21|     1080000|
|         3|       Rahul|        IT| 85000| 2021-06-20|     1020000|
|        10|       Divya|   Finance| 80000| 2020-12-05|      960000|
|         1|        Arun|        IT| 75000| 2023-01-15|      900000|
+----------+------------+----------+------+-----------+------------+
```

Question 8: Data Cleaning and Aggregation



a. Read the CSV file using an appropriate schema and header configuration.

```
df = spark.read.csv(
    "transactions.csv",
    header=True,
    schema=schema
)
```

b. Identify and remove duplicate transactions using a suitable business key.

```
df = df.dropDuplicates(["TransactionID"])
```

c. Handle missing values in required fields and explain your chosen approach.

```
df = df.dropna(subset=[
    "TransactionID", "CustomerID", "Product",
    "Quantity", "UnitPrice"
])
```

d. Convert TransactionDate into a date type and handle invalid dates.

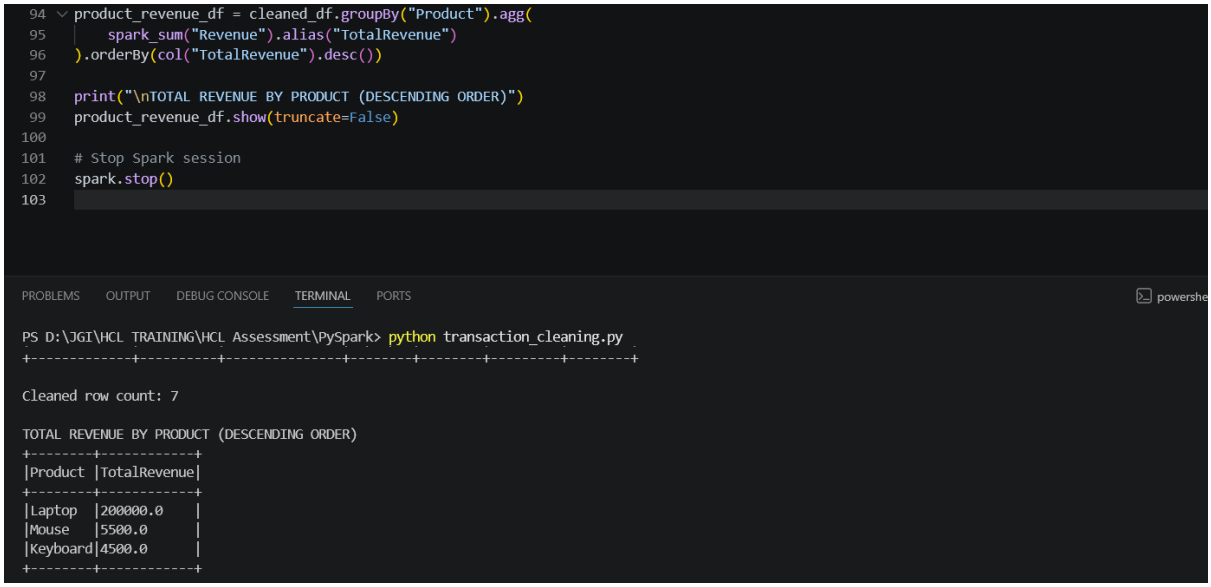
```
df = df.withColumn(
    "TransactionDate",
    coalesce(
        expr("try_to_timestamp(TransactionDate, 'yyyy-MM-dd')"),
        expr("try_to_timestamp(TransactionDate, 'dd/MM/yyyy')"),
        expr("try_to_timestamp(TransactionDate, 'dd-MM-yyyy')")
    ).cast("date")
)
```

e. Calculate revenue using Quantity × UnitPrice.

```
cleaned_df = cleaned_df.withColumn(
    "Revenue",
    col("Quantity") * col("UnitPrice")
)
```

```
df = df.withColumn(
    "Revenue",
    col("Quantity") * col("UnitPrice")
)
```

f. Calculate total revenue by product and display the results in descending order



Question 9: PySpark Joins and Performance

a. Load both datasets into DataFrames

```
    (501, 101, 2000, "2026-10-01"),
    (502, 102, 1500, "2026-10-02"),
    (503, 101, 3000, "2026-10-05")
]

orders_df = spark.createDataFrame(
    orders_data,
    ["OrderID", "CustomerID", "OrderAmount", "OrderDate"]
)

print("\nA. CUSTOMERS DATAFRAME")
customers_df.show()

print("\nB. ORDERS DATAFRAME")
orders_df.show()
```

```
PS D:\JGI\HCL TRAINING\HCL Assessment\PySpark> python transaction_cleaning.py
s://cwiki.apache.org/confluence/display/HADOOP2/WindowsProblems
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/10/29 00:05:35 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable

A. CUSTOMERS DATAFRAME
+----------+------------+---------+
|CustomerID|CustomerName|     City|
+----------+------------+---------+
|       101|        Arun|  Chennai|
|       102|       Priya|Bangalore|
|       103|     Karthik|  Chennai|
|       104|       Divya|   Mumbai|
+----------+------------+---------+
```

Restart Visual Studio Code to apply the latest update.

b. Join the datasets to display orders with customer details.

```
print("\nC. ORDERS WITH CUSTOMER DETAILS")

joined_df = customers_df.join(
    orders_df,
    on="CustomerID",
    how="inner"
)

joined_df.select(
    "CustomerID",
    "CustomerName",
    "City",
    "OrderID",
    "OrderAmount",
    "OrderDate"
).orderBy("OrderID").show(truncate=False)
```

```
PS D:\JGI\HCL TRAINING\HCL Assessment\PySpark> python transaction_cleaning.py


B. ORDERS DATAFRAME
+-------+----------+-----------+----------+
|OrderID|CustomerID|OrderAmount| OrderDate|
+-------+----------+-----------+----------+
|    501|       101|       2000|2026-10-01|
|    502|       102|       1500|2026-10-02|
|    503|       101|       3000|2026-10-05|
+-------+----------+-----------+----------+
```

c. Identify customers who have never placed an order

```
# (c) Find customers who never placed an order
print("\nD. CUSTOMERS WITHOUT ORDERS")

customers_df.join(
    orders_df.select("CustomerID").distinct(),
    on="CustomerID",
    how="left_anti"
).show()
```

```
PS D:\JGI\HCL TRAINING\HCL Assessment\PySpark> python transaction_cleaning.py

C. ORDERS WITH CUSTOMER DETAILS
+----------+------------+---------+-------+-----------+----------+
|CustomerID|CustomerName|City     |OrderID|OrderAmount|OrderDate |
+----------+------------+---------+-------+-----------+----------+
|101       |Arun        |Chennai  |501    |2000       |2026-10-01|
|102       |Priya       |Bangalore|502    |1500       |2026-10-02|
|101       |Arun        |Chennai  |503    |3000       |2026-10-05|
+----------+------------+---------+-------+-----------+----------+
```

d. Calculate the total order value by city

```
print("\nE. TOTAL ORDER VALUE BY CITY")

customers_df.join(
    orders_df,
    on="CustomerID",
    how="left"
).groupBy("City").agg(
    coalesce(
        spark_sum("OrderAmount"),
        lit(0)
    ).alias("TotalOrderValue")
).orderBy("City").show()

# Stop Spark
spark.stop()
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS                    powershell

PS D:\JGI\HCL TRAINING\HCL Assessment\PySpark> python transaction_cleaning.py

D. CUSTOMERS WITHOUT ORDERS
+----------+------------+-------+
|CustomerID|CustomerName|   City|
+----------+------------+-------+
|       103|     Karthik|Chennai|
|       104|       Divya| Mumbai|
+----------+------------+-------+
```

e.Explain when a broadcast join is useful and what conditions make it appropriate.

A broadcast join is useful when one DataFrame is small enough to fit into the memory of each executor. Spark distributes the smaller DataFrame to the worker nodes, reducing data shuffling and improving join performance. It is appropriate when the smaller dataset fits comfortably in the available memory.

f.Explain the difference between transformations and actions, giving one example of each.

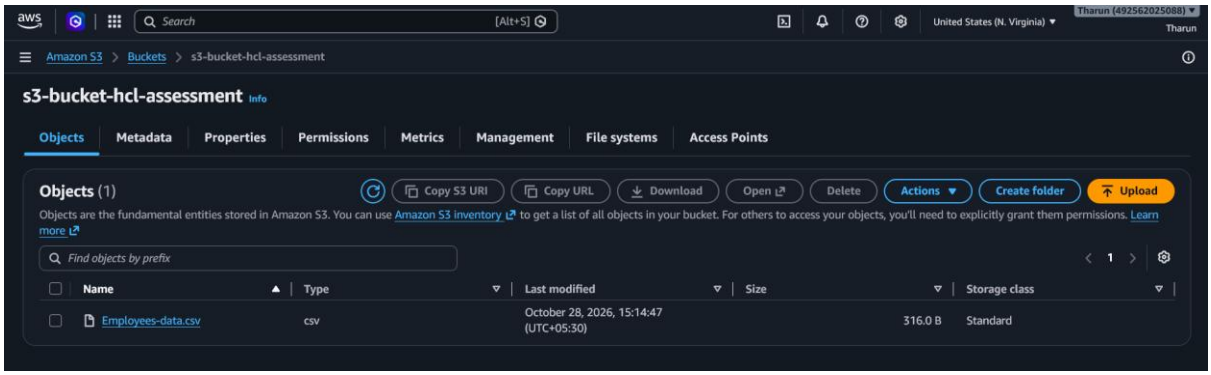
Transformations create a new DataFrame from an existing DataFrame without executing the computation immediately. For example, filter() selects rows that meet a condition.
Actions trigger the execution of Spark operations and return results. For example, count() executes the required operations and returns the number of rows.

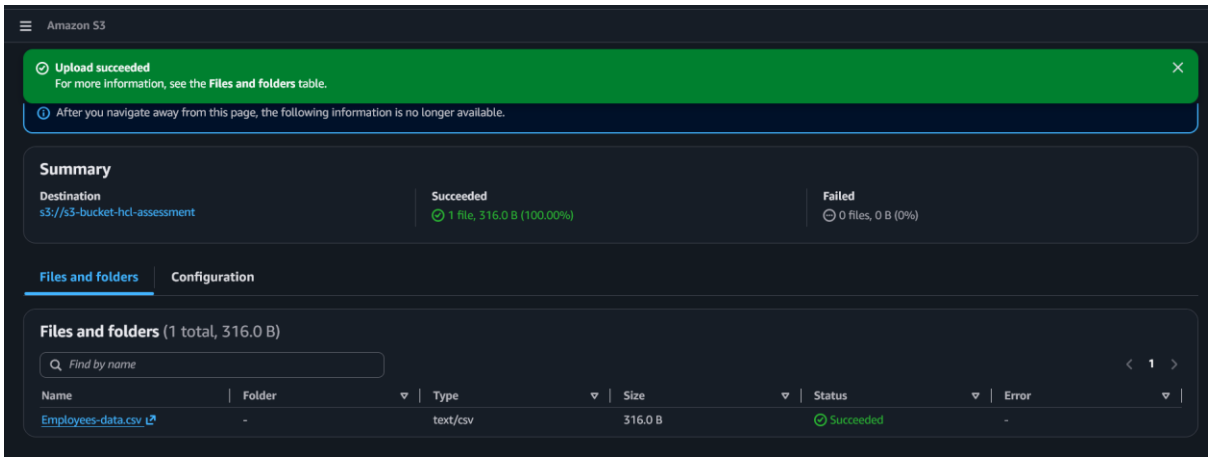
SECTION D

AWS CLOUD SERVICES

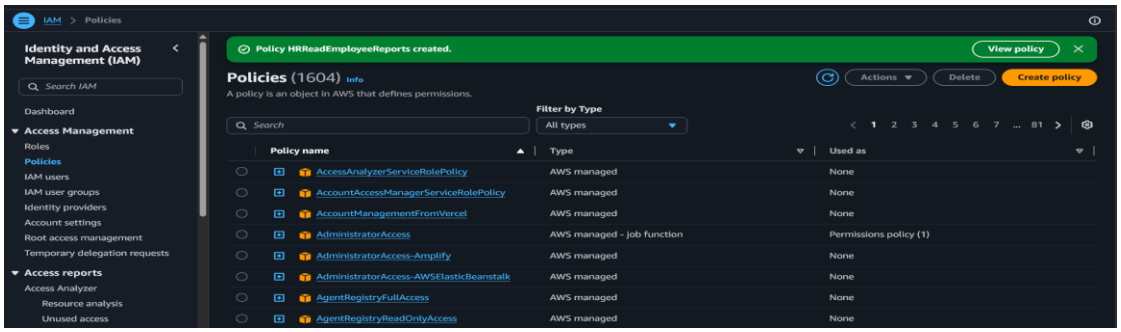
Question 10: S3 and IAM Security

a. Create an S3 bucket with Block Public Access enabled and upload a sample file.





b. Design an IAM policy that grants read access to objects in the designated bucket without granting access to unrelated buckets.



c. Explain how you would verify that an unauthorized user cannot

read the file.

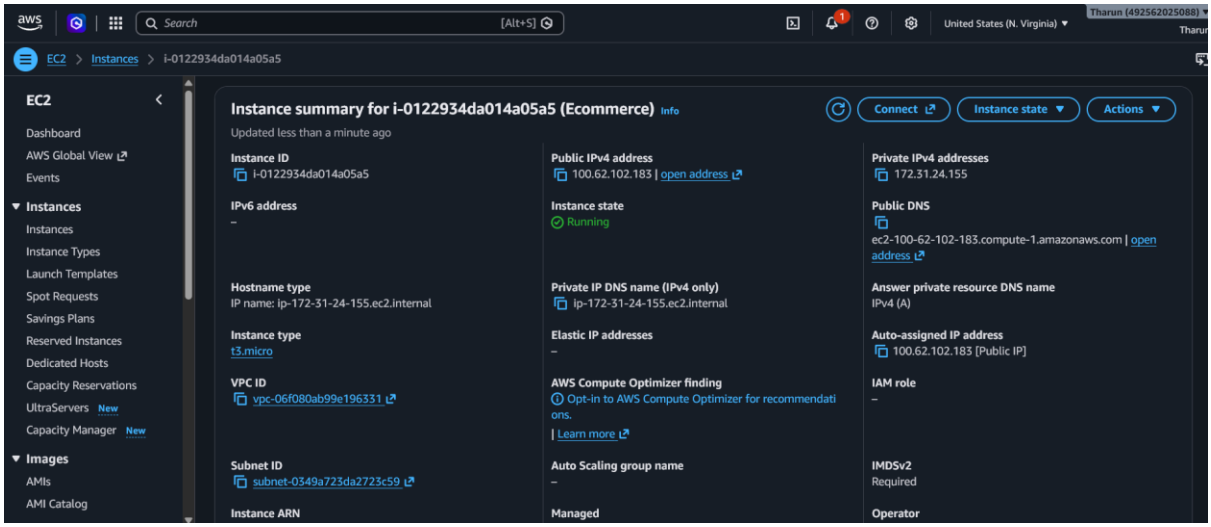
To verify that an unauthorized user cannot read the file, I would use an IAM user who does not have permission to access the S3 bucket. I would sign in as that user and attempt to open or download the uploaded file. AWS should display an “Access Denied” error, confirming that access is restricted to users with the required permissions.

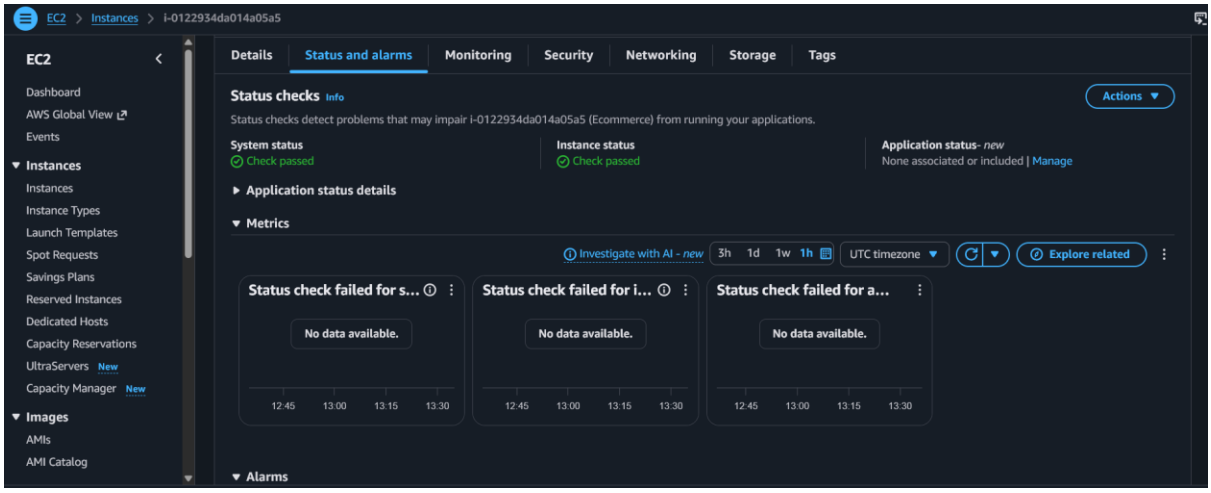
d. Explain how encryption at rest protects the stored data

I would enable default encryption on the S3 bucket to protect the stored employee reports. Amazon S3 uses server-side encryption to convert the stored data into an unreadable format, preventing unauthorized access to the underlying data. When an authorized user requests the file, Amazon S3 automatically decrypts it and provides access to the data.

Question 11: EC2 Troubleshooting and CloudWatch

a. Check the EC2 instance state and status checks.





b. Explain how security groups and route tables can affect application connectivity.

Security Groups: I would check the inbound rules to ensure that HTTP (port 80) or HTTPS (port 443) traffic is allowed. Incorrect rules can prevent users from accessing the web application.

Route Tables: I would verify that the subnet has the correct routes. For a publicly accessible instance, the route table should have a route to an Internet Gateway, and the instance should have a public IP address.

Incorrect security group rules or route table configurations can cause connectivity problems.

c. Explain how to configure a CloudWatch alarm when CPU utilization exceeds 85% for a defined evaluation period.

1. Open the Amazon CloudWatch Console.

2. Navigate to Alarms → All alarms → Create alarm.

3. Click Select metric → EC2 Metrics → Per-Instance Metrics.

4. Select the required EC2 instance and the CPUUtilization metric, then click Select metric.

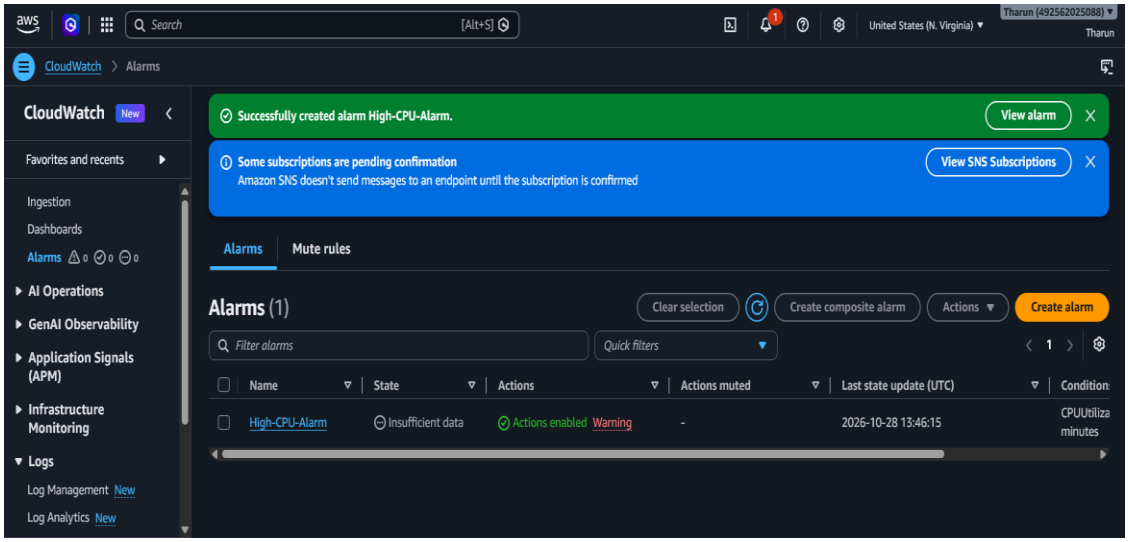
5. Set the statistic to Average and the period to 5 minutes.

6. Set the threshold condition to Greater than 85%.

7. Set the evaluation period to 2 consecutive periods so the alarm triggers when CPU utilization exceeds 85% for two consecutive five-minute periods.

8. Configure an optional Amazon SNS notification to receive an alert by email.

9. Name the alarm High-CPU-Alarm and click Create alarm.



d. Identify one method for investigating the root cause of high CPU utilization.

I would connect to the EC2 instance and use the Linux top command to monitor CPU usage and identify processes consuming excessive resources. I would then investigate the identified processes and check the application logs to determine the cause of high CPU utilization.

Question 12: Event-Driven Processing

a. Identify a suitable service for executing the processing code.

I would use AWS Lambda to execute the processing code. It can validate uploaded CSV files, process the data, and store the results in

a destination S3 bucket without requiring a server to be managed manually.

b. Explain how an S3 upload can trigger processing.

I would configure an S3 event notification to trigger the Lambda function whenever a new CSV file is uploaded. The Lambda function would then automatically read the uploaded file, validate its contents, process the data, and save the results to the destination location.

c. Explain the IAM permissions needed to read the input file and write the results.

I would create an IAM execution role for the Lambda function with the necessary permissions. The role would have s3:GetObject permission to read files from the input bucket and s3:PutObject permission to write processed results to the output bucket. These permissions would be restricted to the required resources to follow the principle of least privilege.

d. Explain how errors and processing logs can be captured.

I would use Amazon CloudWatch Logs to capture the Lambda function's execution logs, including processing details and error messages. If the function fails to process a CSV file, I could examine these logs to identify the cause and troubleshoot the issue.

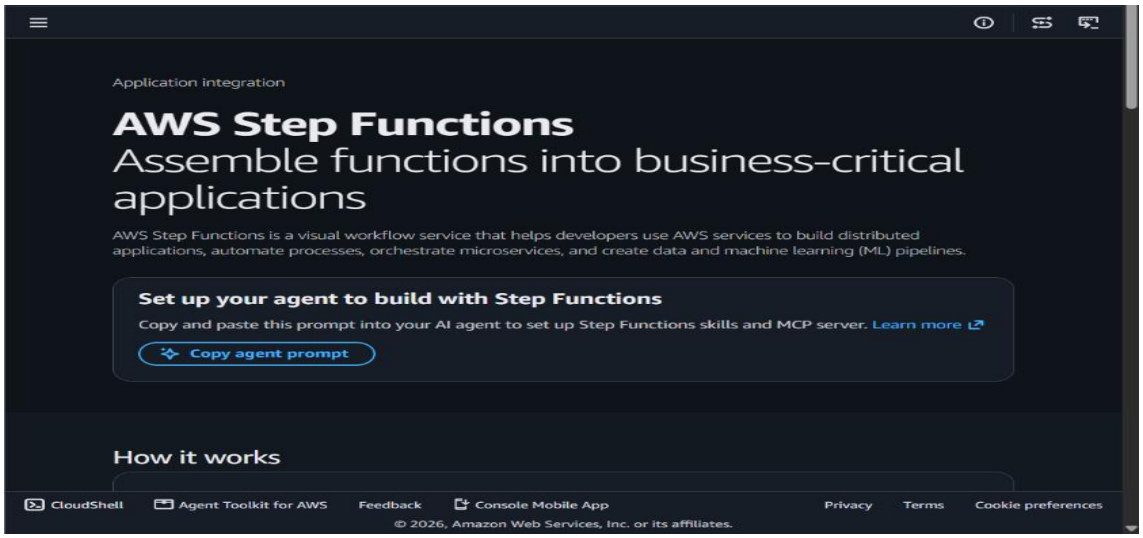
e. Explain how to prevent duplicate processing or recursive triggering.

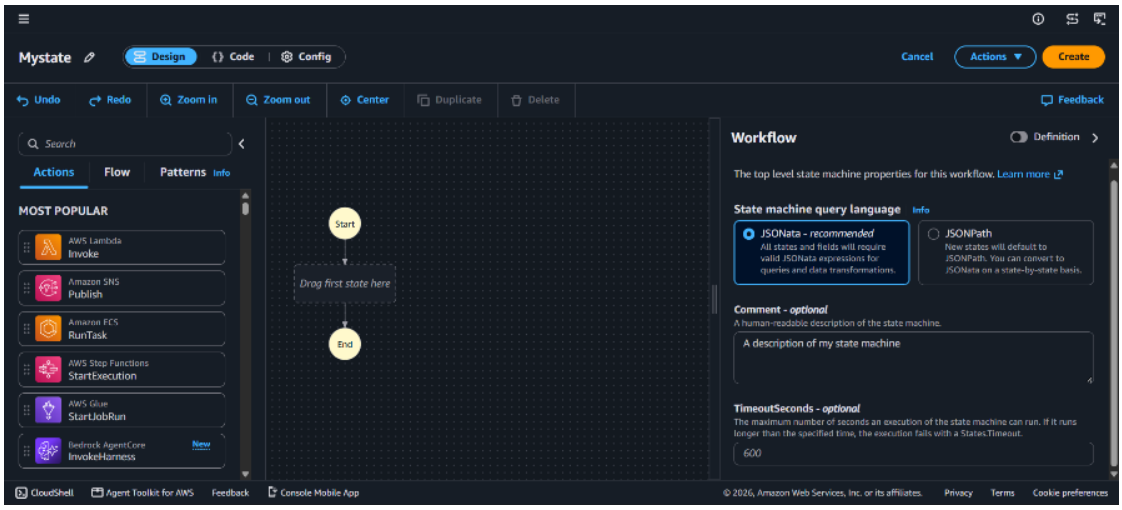
I would use separate S3 buckets for input files and processed results to prevent output uploads from triggering the same Lambda function again. To avoid duplicate processing, I would also track processed file names or version IDs using a service such as Amazon DynamoDB and skip files that have already been processed.

SECTION E

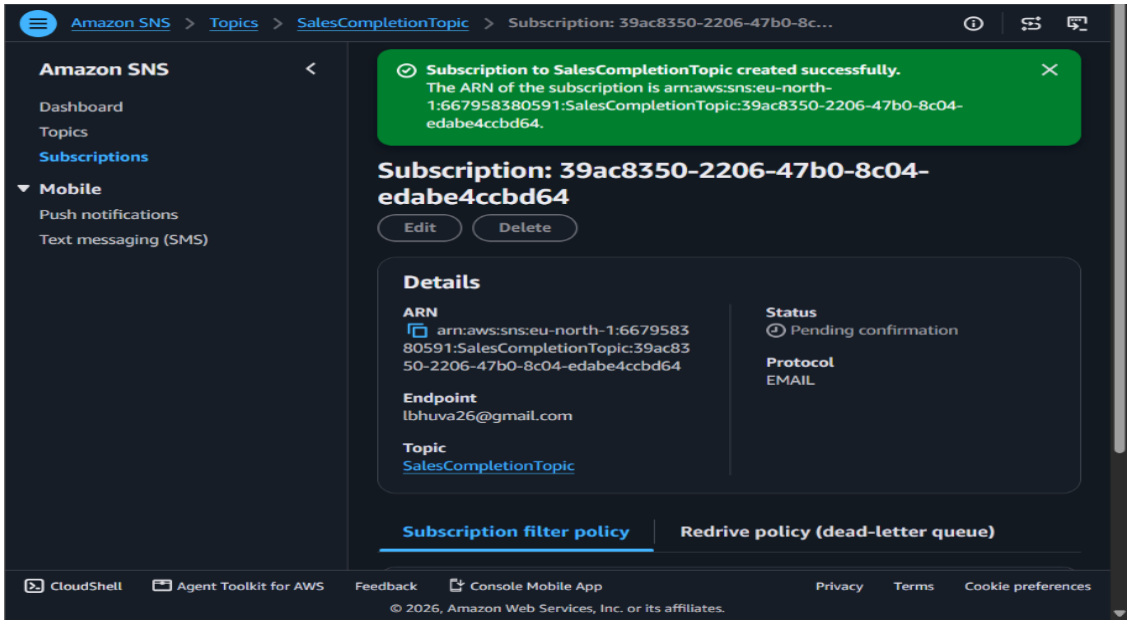
AWS STEP FUNCTIONS

Question 13: State Machine Design









a. Identify suitable states for validation, processing, storage, and notification.

I would use AWS Step Functions with the following states:

- ValidateInput: Validates the sales file using Lambda.
- CheckValidation: Routes valid files to processing and invalid files to failure handling.
- TransformData: Transforms the sales data.
- CalculateSalesByProduct: Calculates total sales for each product.
- StoreResults: Saves results to Amazon S3.
- SendCompletionNotification: Sends a completion notification.
- Success/Failure: Indicates whether the workflow completed or failed.

b. Explain how Retry and Catch can handle transient failures and permanent errors.

The Retry feature automatically retries a task when temporary errors occur, such as network issues or service failures. The Catch feature handles errors that continue after retries by redirecting the workflow to a failure-handling state, where the error can be logged and a notification can be sent.

c. Explain how Retry and Catch can handle transient failures and permanent errors.

The Retry feature automatically retries tasks when temporary errors occur, such as service failures. The Catch feature handles errors that persist after retries by directing the workflow to a failure-handling state.

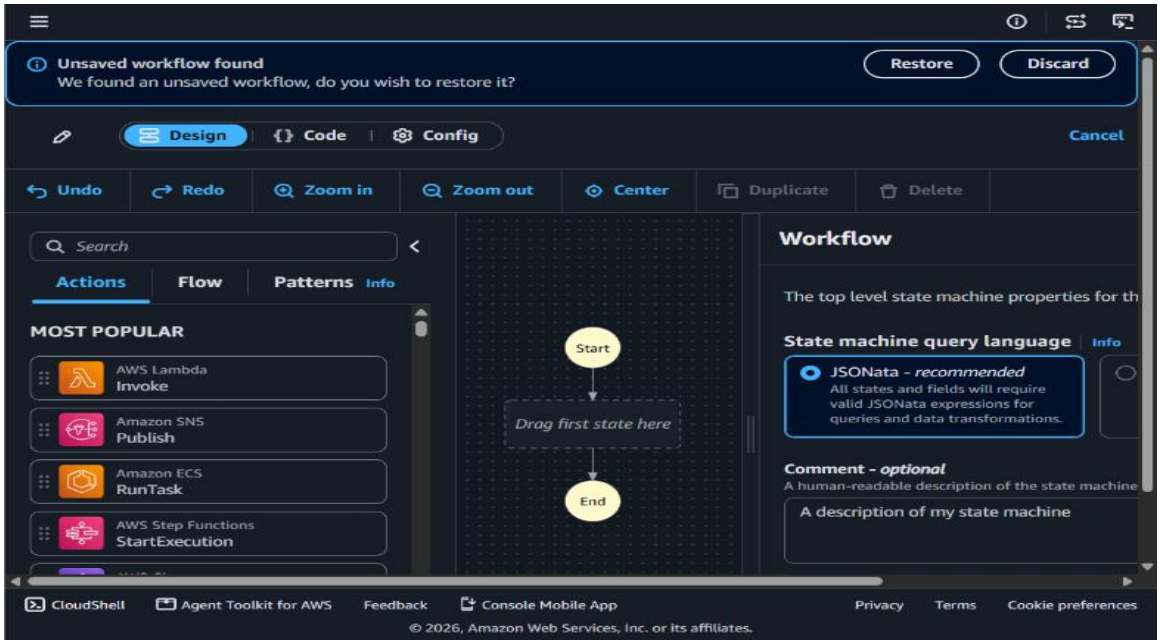
d. Explain how the state machine can invoke Lambda functions or other supported AWS services.

AWS Step Functions uses Task states to invoke Lambda functions or other supported AWS services. These tasks perform operations such as validating files, processing data, storing results in S3, and sending notifications.

e. Explain how execution history helps troubleshoot a failed workflow.

Execution history records the workflow's state transitions, inputs, outputs, and errors. It helps identify the step where the workflow failed and makes troubleshooting easier.

Question 14: Step Functions Implementation



(a) Three sequential tasks

The state machine uses three Task states: ValidateFile validates the input, TransformData transforms the data, and LoadResults loads the processed data into the destination. A Choice state checks the validation result and directs invalid files to a failure state.

(b) Retry policy

The TransformData task uses a Retry policy for temporary Lambda service errors and timeouts. It waits 2 seconds before the first retry, allows up to 3 retries, and doubles the waiting interval after each retry. If all retries fail, the workflow fails because no Catch handler is defined for this task.

AWS Step Functions

(c) Catch transition

The LoadResults task uses Catch with States.ALL to capture task errors and route them to RecordFailure. This state stores the error name and cause in the execution data before directing the workflow to WorkflowFailed. The recorded details are available in the Step Functions execution history.

(d) How data passes between tasks

AWS Step Functions passes the output of each state to the next state as input. The ResultPath field stores each task's result under a specified field, such as $.validation or $.transformation, while preserving the other input data. This allows subsequent tasks to access the earlier results.

(e) Testing successful and failed executions

I would test the state machine with a valid input file and verify that all three tasks execute successfully. I would then test an invalid file to confirm that the workflow stops, simulate a temporary transformation error to verify retries, and simulate a loading failure to verify that the Catch transition records the error. I would review the execution history to confirm the state transitions and outcomes.

SECTION F

MICROSOFT AZURE

Note on Practical Implementation

Due to an account and subscription limitation preventing me from accessing the Microsoft Azure free/student tier, I was unable to execute the configurations through the Azure Portal. Therefore, I have documented the required implementation steps, configurations, and expected outcomes for each question based on Microsoft Azure service documentation. These answers represent the intended implementation procedure; practical execution and screenshots could not be completed because of the current account restriction

(a) Configure a storage account with hierarchical namespace enabled

1. I would open the Microsoft Azure Portal and navigate to Storage accounts → Create.
2. I would select the appropriate subscription and resource group.
3. I would enter a unique storage account name, select the region, and choose Standard General-purpose v2 as the account type.
4. Under the Advanced tab, I would enable Hierarchical namespace to support Azure Data Lake Storage Gen2.
5. Finally, I would review the settings and create the storage account.

This configuration enables hierarchical directory structures required for data lake operations.

(b) Create a filesystem and upload a sample dataset

1. I would open the newly created storage account in the Azure Portal.
2. Under Storage browser or Containers, I would create a new filesystem/container named business-data.
3. I would keep the container private to prevent anonymous access.
4. I would open the container and upload a sample CSV dataset, such as transactions.csv.
5. I would verify that the uploaded file appears in the container.

(c) Explain how Microsoft Entra ID and Azure RBAC manage access

1. I would use Microsoft Entra ID to authenticate users and groups accessing the data.
2. I would assign Azure role-based access control (RBAC) roles at the appropriate storage account or container scope.
3. I would assign Storage Blob Data Reader to users who only need to view or download data, and Storage Blob Data Contributor to users who need to read, write, or delete data.
4. For more detailed permissions on individual directories and files, I would configure access control lists (ACLs).
5. I would follow the principle of least privilege by granting each user only the permissions required for their role.

Microsoft Entra ID provides authentication, while RBAC and ACLs help control data access.

(d) Verify that unauthorized users cannot access the dataset

I would use a separate user account that has not been granted permission to access the dataset.

1. I would attempt to open or download the uploaded CSV file using that account.
2. I would verify that Azure denies the request with an authorization error.
3. This would confirm that access is restricted to authorized users. I would also ensure that anonymous public access is disabled.

Question 16: Azure VM Troubleshooting

(a) Verify the VM's provisioning and power states

1. I would open the Azure Portal and navigate to Virtual machines.
2. I would select the required virtual machine and open its Overview page.
3. I would check the Power state and confirm that the VM is Running.
4. I would check that the provisioning state is Succeeded.
5. If the VM is stopped or deallocated, I would start it. If provisioning has failed, I would examine the error details and Activity Log.

The provisioning state indicates whether the latest resource operation succeeded, while the power state indicates the VM's current operating state.

(b) Explain how Network Security Group rules can prevent connectivity

Network Security Groups (NSGs) control inbound and outbound network traffic using security rules. Incorrect rules may block the connection even when the VM is running.

1. I would open the VM's Networking settings and inspect its NSG rules.

2. I would verify that the required port is allowed, such as TCP port 3389 for Windows RDP or port 22 for Linux SSH.
3. I would check the source IP address, rule priority, and any conflicting deny rules.
4. I would also check the NSGs associated with both the VM's network interface and subnet, along with the operating system's firewall.

(c) Explain how subnet and routing configuration affect connectivity

1. I would verify that the VM's network interface is connected to the correct virtual network and subnet.
2. I would inspect the subnet's route table and effective routes to identify incorrect or conflicting routes.
3. I would check whether user-defined routes, network peering, or VPN configuration is preventing traffic from reaching the VM.
4. For private connectivity, I would ensure that the required network routes and access rules are configured correctly.

Incorrect routes or subnet configurations can prevent traffic from reaching the VM even if its power state is healthy.

(d) Recommend a secure troubleshooting method

I would use Azure Bastion to connect securely to the VM without exposing the VM itself to the public internet.

1. I would configure Azure Bastion within the virtual network, subject to subscription availability and permissions.
2. I would connect to the VM through the Azure Portal using its private IP address.

3. I would avoid assigning a public IP address to the VM solely for troubleshooting and would not open RDP or SSH ports to all internet addresses.
4. If the connection still failed, I would use Azure Network Watcher Connection Troubleshoot to investigate network connectivity and inspect the relevant NSG and route configurations.
5. If necessary, I would use supported recovery tools such as Run Command or Serial Console to investigate operating system services without broadly exposing the VM.

Azure Bastion supports secure RDP and SSH access to VMs without requiring a public IP address on the target VM.

Question 17: Azure Data Integration

(a) Select an appropriate service to orchestrate data movement

I would use Azure Data Factory (ADF) to orchestrate data movement. It allows me to create pipelines that copy, transform, and organize customer transaction data for analytics.

(b) Explain how to copy the CSV file into Azure Data Lake Storage Gen2

1. I would create an Azure Data Factory instance in the Azure Portal.
2. I would create a linked service connecting ADF to the location where the CSV file arrives.
3. I would create another linked service for the destination Azure Data Lake Storage Gen2 account, preferably using a managed identity.

4. I would create a pipeline containing a Copy Data activity, configure the source dataset as CSV, and select the destination filesystem and folder.
5. I would run the pipeline and verify that the file has been copied successfully to the data lake.

Azure Data Factory supports Copy activity for moving data to Azure Data Lake Storage Gen2.

(c) Describe how to organize files by ingestion date

I would organize the uploaded files into folders based on their ingestion date. For example:

```
business-data/
    raw/
        transactions/
            ingestion_date=2026-10-29/
                transactions.csv
            ingestion_date=2026-10-30/
                transactions.csv
```

I would configure the pipeline to generate the ingestion-date folder dynamically using the processing or ingestion timestamp. This structure makes it easier to locate data for a specific date, maintain historical records, and process daily files independently.

(d) Explain how to handle malformed records and missing values

I would validate the CSV schema, column names, data types, and mandatory fields before processing the records. Records containing invalid dates, incorrect data types, or missing required values would be rejected or stored in a separate error folder for review. Optional missing values could be replaced with NULL or suitable default

values according to business requirements. I would also maintain error logs to track rejected records. Azure Data Factory Copy activity supports fault-tolerance options for handling and logging certain incompatible records.

(e) Explain how to monitor pipeline execution and validate data completeness

1. I would open the Monitor section in Azure Data Factory and review pipeline and activity execution statuses.
2. I would examine errors, execution duration, rows read, rows written, and any skipped records.
3. I would compare the source and destination row counts and verify the required columns, data types, and mandatory fields.
4. I would configure appropriate alerts for failed pipeline executions and investigate any discrepancies before making the data available for analytics.

Azure Data Factory provides pipeline monitoring and activity-level execution details to help identify failures and verify data movement.

.